
Chapter Four

Software Project Estimation

Introduction

- **Estimation** is attempt to determine how much money, effort, resources & time it will take to build a specific software based system or project.
- Effective software project estimation is one of the most challenging and important activities in software development.
- Proper project planning and control is not possible without a sound and reliable estimate.
- Under-estimating a project leads to under-staffing it
 - » resulting in staff burnout
- Under-scoping the quality assurance effort
 - » running the risk of low quality deliverables and
- Setting too short schedule
 - » resulting in loss of credibility as deadlines are missed.

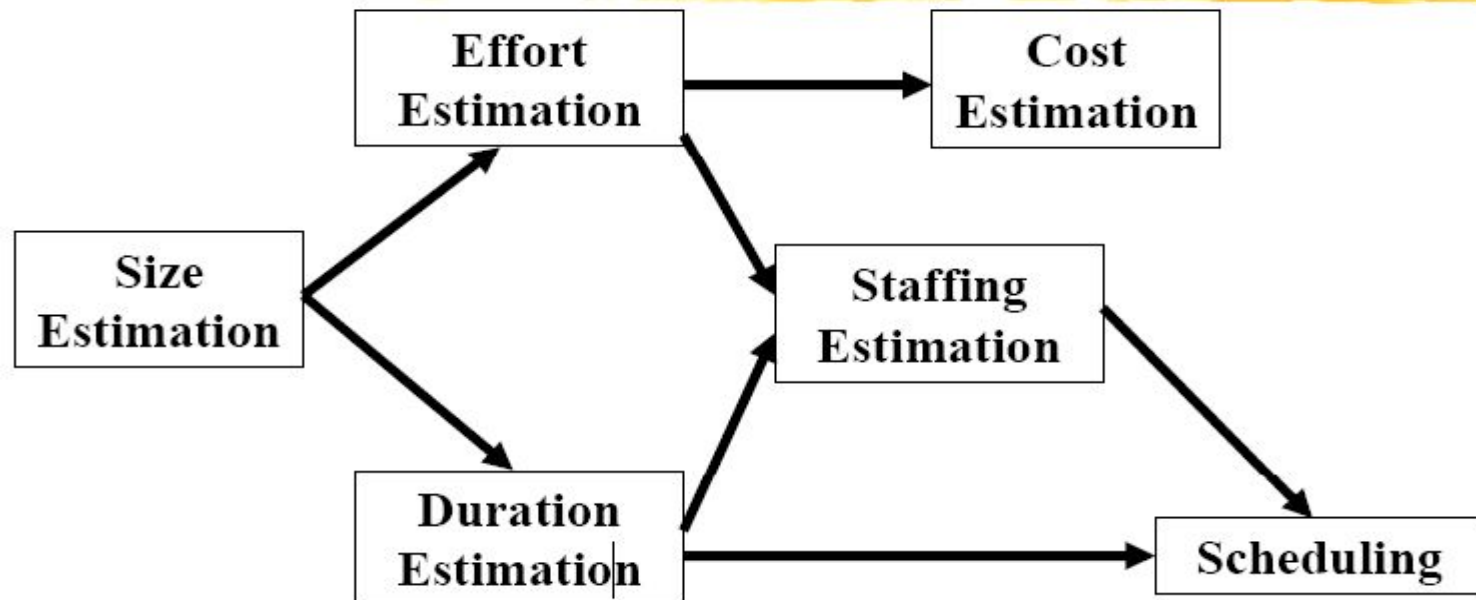
Cont'd

- Over-estimating a project can be just about as bad for the organization.
- The project is then likely to cost more than it should
 - » a negative impact on the bottom line
- Take longer to deliver than necessary
 - » resulting in lost opportunities and
- Delay the use of your resources on the next project.

Cont'd

- Estimation includes:

- » **Project size:** What will be problem complexity in terms of the effort and time required to develop the product?
- » **Cost:** How much is it going to cost to develop the project?
- » **Duration:** How long is it going to take to complete development?
- » **Effort:** How much effort would be required?



Metrics for Software Project Size Estimation

- Currently two metrics are popularly being used widely to estimate size:
 - » lines of code (LOC) and
 - » function point (FP).
- The usage of each of these metrics in project size estimation has its own advantages and disadvantages.

Lines of Code (LOC)

- LOC is the simplest among all metrics available to estimate project size.
- It is very popular because it is the simplest to use.
 - » Using this metric, the project size is estimated by counting the number of source instructions in the developed program.
 - » While counting the number of source instructions, lines used for commenting the code and the header lines should be ignored.
- In order to estimate the LOC count at the beginning of a project, project managers usually divide the problem into modules, and each module into submodules and so on, until the sizes of the different leaf-level modules can be approximately predicted.

Shortcomings of LOC

- LOC gives a numerical value of problem size that can vary widely with individual coding style
- It should consider the local effort needed to specify, design, code, test, etc. and not just the coding effort.
- LOC measure correlates poorly with the quality and efficiency of the code.
- LOC metric penalizes use of higher-level programming languages, code reuse, etc.
- LOC metric measures the lexical complexity of a program and does not address the more important but subtle issues of logical or structural complexities.
- It is very difficult to accurately estimate LOC in the final product from the problem specification.

Function Point (FP)

- Function point metric was proposed by Albrecht [1983].
- The conceptual idea behind the function point metric is that the size of a software product is directly dependent on the number of different functions or features it supports.
- Each function when invoked reads some input data and transforms it to the corresponding output data.
- The computation of the number of input and the output data values to a system gives some indication of the number of functions supported by the system.
- Albrecht postulated that in addition to the number of basic functions that a software performs, the size is also dependent on the number of files and the number of interfaces.

Cont'd

- Besides using the number of input and output data values, function point metric computes the size of a software product (in units of functions points or FPs) using three other characteristics of the product as shown in the following expression.
- Function point is computed in two steps. The first step is to compute the unadjusted function point (UFP).
- **$$\text{UFP} = (\text{Number of inputs}) * 4 + (\text{Number of outputs}) * 5 + (\text{Number of inquiries}) * 4 + (\text{Number of files}) * 10 + (\text{Number of interfaces}) * 7$$**
- The size of a product in function points (FP) can be expressed as the weighted sum of these five problem characteristics.

Cont'd

- The weights associated with the five characteristics were proposed empirically and validated by the observations over many projects.
- **Number of inputs:**
 - » Each data item input by the user is counted.
 - » Data inputs should be distinguished from user inquiries.
 - » Inquiries are user commands such as print-account-balance.
 - » A group of related inputs are considered as a single input.
- **Number of outputs:**
 - » The outputs considered refer to reports printed, screen outputs, error messages produced, etc.

Cont'd

- **Number of inquiries:**

- » It is the number of distinct interactive queries which can be made by the users.

- **Number of files:**

- » Each logical file is counted. A logical file means groups of logically related data.

- **Number of interfaces:**

- » Here the interfaces considered are the interfaces used to exchange information with other systems.

Counting Function Points

- Complexity level of each parameter is graded as low, average and high.
- The weight for the different parameter can be compute based on the following table:

Functional Units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
External logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Cont'd

Functional Units	Count Complexity			Complexity Totals	Functional Unit Totals
External Inputs (EIs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External Outputs (EOs)		Low x 4	=		
		Average x 5	=		
		High x 7	=		
External Inquiries (EQs)		Low x 3	=		
		Average x 4	=		
		High x 6	=		
External logical Files (ILFs)		Low x 7	=		
		Average x 10	=		
		High x 15	=		
External Interface Files (EIFs)		Low x 5	=		
		Average x 7	=		
		High x 10	=		
Total Unadjusted Function Point Count					

FP Adjustment

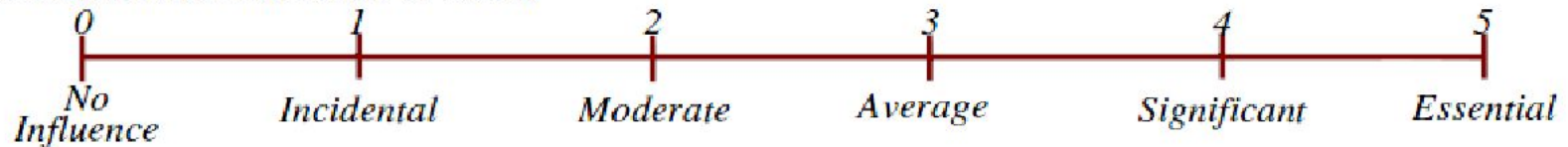
- Organizations that use function point method develop a criterion for determining whether a particular entry is Low, Average or High.
- Nonetheless, the determination of complexity is somewhat subjective.

$$FP = UFP * CAF$$

- Where CAF is a complexity adjustment factor and is equal to **$[0.65 + 0.01 * \sum F_i]$** .
- The F_i ($i=1$ to 14) are the degree of influence and are based on responses to questions.

Cont'd

Rate each factor on a scale of 0 to 5.



Number of factors considered (F_i)

1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

Example

- Consider a project with the following parameters:
 - i. External Inputs: 10 with low complexity, 15 with average complexity and 17 with high complexity.
 - ii. External Output: 6 with low complexity and 13 with high complexity.
 - iii. External Inquiries: 3 with low complexity, 4 with average complexity. and 2 with high complexity.
 - iv. Internal Logical Files: 2 with average complexity and 1 with high complexity.
 - v. External Interface Files: 9 with low complexity.

Cont'd

- In addition to above, the system requires:
 - i. Significant data communication
 - ii. Performance is very critical
 - iii. Designed code may be moderately reusable
 - iv. System is not designed for multiple installation in different organizations.
 - v. Other complexity adjustment factors are treated as average.
- Compute the function points for the project.

Cont'd

Functional Units	Count	Complexity		Complexity Totals	Functional Unit Totals
External Inputs (EIs)	10	Low x 3	=	30	192
	15	Average x 4	=	60	
	17	High x 6	=	102	
External Outputs (EOs)	6	Low x 4	=	24	115
	0	Average x 5	=	0	
	13	High x 7	=	91	
External Inquiries (EQs)	3	Low x 3	=	9	37
	4	Average x 4	=	16	
	2	High x 6	=	12	
External logical Files (ILFs)	0	Low x 7	=	0	35
	2	Average x 10	=	20	
	1	High x 15	=	15	
External Interface Files (EIFs)	9	Low x 5	=	45	45
	0	Average x 7	=	0	
	0	High x 10	=	0	
Total Unadjusted Function Point Count					424

Cont'd

- $\sum_{i=1}^{14} F_i = 3+4+3+5+3+3+3+3+3+3+2+3+0+3 = 41$

$$\begin{aligned} \text{CAF} &= (0.65 + 0.01 * \sum F_i) \\ &= (0.65 + 0.01 * 41) \\ &= 1.06 \end{aligned}$$

$$\begin{aligned} \text{FP} &= \text{UFP} * \text{CAF} \\ &= 424 * 1.06 \\ &= 449.44 \approx \mathbf{449} \end{aligned}$$

Feature Point Metric

- Feature point metric incorporates an extra parameter algorithm complexity.
- This parameter ensures that the computed size using the feature point metric reflects the fact that the more is the complexity of a function, the greater is the effort required to develop it and therefore its size should be larger compared to simpler functions.

Estimation Techniques

- Estimation of various project parameters is a basic project planning activity.
- The important project parameters that are estimated include: ***project size, effort required to develop the software, project duration, and cost.***
- These estimates not only help in quoting the project cost to the customer, but are also useful in resource planning and scheduling.
- There are three broad categories of estimation techniques:
 - » Empirical estimation techniques
 - » Heuristic techniques
 - » Analytical estimation techniques

Empirical Estimation Techniques

- Empirical estimation techniques are based on making an educated guess of the project parameters.
- Although empirical estimation techniques are based on common sense, different activities involved in estimation have been formalized over the years.
- Two popular empirical estimation techniques are:
 - » Expert judgment technique and
 - » Delphi cost estimation.

Expert Judgment Technique

- In this approach, an expert makes an educated guess of the problem size after analyzing the problem thoroughly and estimates the cost of the different components (i.e. modules or subsystems) of the system and then combines them to arrive at the overall estimate.
- Problems
 - » Human errors and individual bias.
 - » Expert may overlook some factors by mistake.
 - » Lack of experience and knowledge.
- A more refined form of expert judgment is the estimation made by group of experts.
 - » It minimizes factors such as individual oversight, lack of familiarity with a particular aspect of a project, personal bias, and the desire to win contract through overly optimistic estimates.

Delphi Cost Estimation

- Delphi estimation is carried out by a team of experts and a coordinator.
- In this approach, the coordinator provides each estimator with a copy of SRS document to make initial estimate.
- In their estimates, the estimators mention any unusual characteristic of the product which has influenced his estimation.
- The coordinator prepares and distributes the summary of the responses of all the estimators, and includes any unusual rationale noted by any of the estimators and asks the estimators to re-estimate.

Cont'd

- This process is iterated for several rounds.
- However, no discussion among the estimators is allowed during the entire estimation process to avoid any biasing.
- After the completion of several iterations of estimations, the coordinator takes the responsibility of compiling the results and preparing the final estimate.

Heuristic Techniques

- Heuristic techniques assume that the relationships among the different project parameters can be modeled using suitable mathematical expressions.
- Once the basic (independent) parameters are known, the other (dependent) parameters can be easily determined by substituting the value of the basic parameters.
- Heuristic estimation model can be divided into the following two classes:
 - » Single variable model and
 - » Multi variable model.

Single Variable Model

- Single variable estimation models provide a means to estimate the desired characteristics of a problem, using some previously estimated basic (independent) characteristic of the software product such as its size.
- A single variable estimation model takes the following form:
$$\text{Estimated Parameter} = c^1 * e^{d1}$$
- In the above expression, **e** is the characteristic of the software which has already been estimated (independent variable).

Cont'd

- Estimated Parameter is the dependent parameter to be estimated which could be effort, project duration, staff size, etc.
- **c1** and **d1** are constants.
- The values of the constants **c1** and **d1** are usually determined using data collected from past projects (historical data).
- The basic COCOMO model is an example of single variable cost estimation model.

COCOMO (COSt COnstructive MOdel)

- In COCOMO, projects are categorized into three types:
 - » Organic project
 - » Embedded projects
 - » Semi-detached projects
- **Organic projects:**
 - » are in areas in which the organization has considerable experience, and requirements are less stringent (less strict).
 - » Usually developed by a small team.
 - » E.g. simple business systems, simple inventory management systems.
 - » if the project deals with developing a well understood application program, the size of the development team is reasonably small, and the team members are experienced in developing similar types of projects.

Cont'd

- **Semi-detached projects:**
 - » if the development consists of a mixture of experienced and inexperienced staff.
 - » Team members may have limited experience on related systems but may be unfamiliar with some aspects of the system being developed.
 - » Includes developing a new operating system (OS), a database management system (DBMS), a complex inventory management system.
- **Embedded projects:**
 - » if the software being developed is strongly coupled to complex hardware, or if the stringent regulations on the operational procedures exist.
 - » are in areas in which the organization has little experience, and stringent requirements for such aspects as interfacing and reliability.
 - » E.g. real-time command systems.

Basic COCOMO Model

- The basic COCOMO model gives an approximate estimate of the project parameters.
- The basic COCOMO estimation model is given by the following expressions:
 - » **Effort = $a1 \times (KLOC)^{a2}$ PM**
 - » **Tdev = $b1 \times (Effort)^{b2}$ Months**
- Where,
 - » KLOC is the estimated size of the software product expressed in Kilo Lines of Code,
 - » **a1, a2, b1, b2** are constants for each category of software products,
 - » Tdev is the estimated time to develop the software, expressed in months,
 - » Effort is the total effort required to develop the software product, expressed in person months (PMs).

Cont'd

- **Estimation of development effort**
- For the three classes of software products, the formulas for estimating the effort based on the code size are shown below:
 - » Organic : **Effort = $2.4(KLOC)^{1.05}$ PM**
 - » Semi-detached : **Effort = $3.0(KLOC)^{1.12}$ PM**
 - » Embedded : **Effort = $3.6(KLOC)^{1.20}$ PM**
- **Estimation of development time**
 - » Organic : **Tdev = $2.5(Effort)^{0.38}$ Months**
 - » Semi-detached : **Tdev = $2.5(Effort)^{0.35}$ Months**
 - » Embedded : **Tdev = $2.5(Effort)^{0.32}$ Months**

Staff Size and Productivity

- When effort and development time are known the average staff size to complete the project may be calculated as:

$$\text{Average staff size (SS)} = E/D \text{ Persons}$$

- When project size is known, the productivity level may be calculated as:

$$\text{Productivity (P)} = KLOC/E \text{ (in KLOC/PM)}$$

Example

- Suppose that a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three modes (organic, semidetached and embedded).

The basic COCOMO equation take the form:

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (KLOC)^{d_b}$$

(ii) Semidetached mode

$$E = 3.0(400)^{1.12} = 2462.79 \text{ PM}$$

$$D = 2.5(2462.79)^{0.35} = 38.45 \text{ PM}$$

Estimated size of the project = 400 KLOC

(i) Organic mode

$$E = 2.4(400)^{1.05} = 1295.31 \text{ PM}$$

$$D = 2.5(1295.31)^{0.38} = 38.07 \text{ PM}$$

(iii) Embedded mode

$$E = 3.6(400)^{1.20} = 4772.81 \text{ PM}$$

$$D = 2.5(4772.8)^{0.32} = 38 \text{ PM}$$

Multivariable cost estimation model

$$\text{Estimated Resource} = c^1 * e^{1d1} + c^2 * e^{2d2} + \dots$$

- Where **e1**, **e2**, ... are the basic (independent) characteristics of the software already estimated, and **c1**, **c2**, **d1**, **d2**, ... are constants.
- Multivariable estimation models are expected to give more accurate estimates compared to the single variable models.
- The independent parameters influence the dependent parameter to different extents. This is modeled by the constants **c1**, **c2**, **d1**, **d2**,
- Values of these constants are usually determined from historical data.
- The intermediate COCOMO model can be considered to be an example of a multivariable estimation model.

Intermediate COCOMO

- There are 15 different attributes, called ***cost driver attributes***, that determine the multiplying factors.
- These factors depend on *product*, *computer*, *personnel*, and *technology (project)* attributes.
- Each cost driver has a rating scale (very low, low, nominal, high, very high, extra high).

Multipliers of Cost Drivers

Cost Drivers	Rating				
	Very low	Low	Nominal	High	Very high
Product Attributes					
RELY, required reliability	0.75	0.88	1.00	1.15	1.40
DATA, database size	-	0.94	1.00	1.08	1.16
CPLX, product complexity	0.70	0.85	1.00	1.15	1.30
Computer Attributes					
TIME, execution time constraint	-	-	1.00	1.11	1.30
STOR, main storage constraint	-	-	1.00	1.06	1.21
VIRT, virtual machine volatility	-	0.87	1.00	1.15	1.30
TURN, computer turnaround time	-	0.87	1.00	1.07	1.15
Personnel Attributes					
ACAP, analyst capability	1.46	1.19	1.00	0.86	0.71
AEXP, application experience	1.29	1.13	1.00	0.91	0.82
PCAP, programmer capability	1.42	1.17	1.00	0.86	0.70
VEXP, virtual machine experience	1.21	1.10	1.00	0.90	-
LEXP, programming language experience	1.14	1.07	1.00	0.95	-
Project Attributes					
MODP, modern programming practices	1.24	1.10	1.00	0.91	0.82
TOOL, use of software tools	1.24	1.10	1.00	0.91	0.83
SCHED, development schedule	1.23	1.08	1.00	1.04	1.10

Cont'd

Intermediate COCOMO equations

$$E = a_i (KLOC)^{b_i} * EAF$$

$$D = c_i (E)^{d_i}$$

Project	a_i	b_i	c_i	d_i
Organic	3.2	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Steps for applying COCOMO

- Determine the initial estimate (nominal estimate) of the development effort from the estimate of thousands of delivered lines of source code (KDLOC).
 - $E_i = a * (KDLOC)^b$
- Determine a set of 15 multiplying factors from different attributes of the project, and compute the effort adjustment factor (EAF).
 - EAF = product of all the 15 multiplying factors.
- Compute the total effort by multiplying the initial effort with the effort adjustment factor.
 - Total effort = $E_i * EAF$

Exercise

- Assume 4 modules are to be contained in a project. Assume the project is organic.
- The modules are $M1 = 0.5$ KDLOC, $M2 = 0.5$ KDLOC, $M3 = 0.3$ KDLOC, and $M4 = 0.7$ KDLOC.
- And the multiplying factors are:
 - » RELY, required reliability is very high
 - » CPLX, product complexity is.... high
 - » STOR, main storage constraint is ... high
 - » Others are ... nominal
- Calculate: i) The initial effort
ii) The total effort.

Analytical Estimation Techniques

- Analytical estimation techniques derive the required results starting with basic assumptions regarding the project.
- Thus, unlike empirical and heuristic techniques, analytical techniques do have scientific basis.
- Halstead's software science is an example of an analytical technique.
 - » It can be used to derive some interesting results starting with a few simple assumptions.
 - » Halstead's software science is especially useful for estimating software maintenance efforts. In fact, it outperforms both empirical and heuristic techniques when used for predicting software maintenance efforts.

Halstead's Software Science

- Halstead's software science is an analytical technique to measure size, development effort, and development cost of software products.
- Halstead used a few primitive program parameters to develop the expressions for over all program length, potential minimum value, actual volume, effort, and development time.
- For a given program, let:
 - » n_1 be the number of unique operators used in the program,
 - » n_2 be the number of unique operands used in the program,
 - » N_1 be the total number of operators used in the program,
 - » N_2 be the total number of operands used in the program.

Cont'd...Different Parameters

- **Length and Vocabulary**

- » The length of a program is total usage of all operators and operands in the program. Thus, length $N = N_1 + N_2$.
- » Program vocabulary is the number of unique operators and operands used in the program. Thus, program vocabulary $\eta = \eta_1 + \eta_2$.

- **Program Volume**

- » $V = N \log_2 \eta$
- » Here the program volume V is the minimum number of bits needed to encode the program

Cont'd

- **Potential Minimum Volume**

- » The potential minimum volume V^* is defined as the volume of most concise program in which a problem can be coded.
- » The minimum volume is obtained when the program can be expressed using a single source code instruction., say a function call like `foo( ) ;`.
- » In other words, the volume is bound from below due to the fact that a program would have at least two operators and no less than the requisite number of operands.
- » Thus, if an algorithm operates on input and output data $d_1, d_2, \dots d_n$, the most concise program would be $f(d_1, d_2, \dots d_n)$; for which $\eta_1 = 2, \eta_2 = n$. Therefore, $V^* = (2 + \eta_2)\log_2(2 + \eta_2)$.

Cont'd

- The program level L is given by $L = V^*/V$.
- » The concept of program level L is introduced in an attempt to measure the level of abstraction provided by the programming language.
- » The above result implies that the higher the level of a language, the less effort it takes to develop a program using that language.

Cont'd

- **Effort and Time**

- » The effort required to develop a program can be obtained by dividing the program volume with the level of the programming language used to develop the code.
- » Thus, effort $E = V/L$,
- » Thus, the programming effort $E = V^2/V^*$ (since $L = V^*/V$) varies as the square of the volume.
- The programmer's time $T = E/S$, where S the speed of mental discriminations. The value of S has been empirically developed from psychological reasoning, and its recommended value for programming applications is 18.

Cont'd

- **Length of Program**

- In terms of unique operators and operands

- » It can be assumed that any program of length N consists of N/η unique strings of length η .

- » Now, it is standard combinatorial result that for any given alphabet of size K , there are exactly K^r different strings of length r .

- Thus,

- » $N/\eta \leq \eta^\eta$ Or, $N \leq \eta^{\eta+1}$

Cont'd

- Since operators and operands usually alternate in a program, the upper bound can be further refined into $N \leq \eta_1^{\eta_1} \eta_2^{\eta_2}$. Also, N must include not only the ordered set of n elements, but it should also include all possible subsets of that ordered sets, i.e. the power set of N strings
- Therefore,
 - » $2^N = \eta_1^{\eta_1} \eta_2^{\eta_2}$ Or, taking logarithm on both sides,
 - » $N = \log_2 \eta_1 + \log_2 (\eta_1^{\eta_1} \eta_2^{\eta_2})$
 - » So we get, $N = \log_2 (\eta_1^{\eta_1} \eta_2^{\eta_2})$ approx
 - » **$N = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$**

Example

- Let us consider the following C program:

```
main( )  
{  
int a, b, c, avg;  
scanf("%d %d %d", &a, &b, &c);  
avg = (a+b+c)/3;  
printf("avg = %d", avg);  
}
```

The unique operators are:

main, (), {}, int, scanf, &, “,”, “;”, =, +, /, printf

Cont'd

- The unique operands are:

a, b, c, &a, &b, &c, a+b+c, avg, 3, “%d %d %d”, “avg = %d”

Therefore,

$$\eta^1 = 12, \eta^2 = 11$$

$$\text{Estimated Length} = (12 * \log 12 + 11 * \log 11)$$

$$= (12 * 3.58 + 11 * 3.45)$$

$$= (43 + 38) = 81$$

$$\text{Volume} = \text{Length} * \log(23)$$

$$= 81 * 4.52$$

$$= 366$$